

DOCUMENTACIÓN TÉCNICA

GITHUB WRAPPED

Versión 1.0.0

Estado: Final

Autor(a): Emily Monterrosa

Barranquilla, Colombia – 01 septiembre de 2026

1. INTRODUCCIÓN Y CONTEXTO DEL PRODUCTO

1.1 Problema que resuelve

Un desarrollador que usa GitHub de forma habitual genera una enorme cantidad de datos crudos: commits, repositorios, lenguajes utilizados, horarios de trabajo. Esta información existe, pero está dispersa y no es legible en términos de comportamiento: GitHub no responde preguntas como «a qué hora soy más productivo», «cómo ha cambiado mi stack en el último año» o «cuál ha sido mi racha más larga de commits». GitHub Wrapped existe para cerrar esa brecha entre datos disponibles e insights comprensibles.

1.2 Propuesta de valor

El valor del producto no está en mostrar números —eso ya lo hace el perfil de GitHub y herramientas como git log— sino en la capa de interpretación: tomar métricas estructuradas y convertirlas en afirmaciones narrativas sobre hábitos de desarrollo, presentadas en una experiencia visual, personalizada y compartible, inspirada en el formato de Spotify Wrapped.
Esto implica que el proyecto tiene dos mitades igualmente importantes: una mitad analítica (el Analytics Engine, que calcula métricas exactas y verificables) y una mitad narrativa (el Insights Engine, que traduce esas métricas a lenguaje natural). El documento detalla por qué estas dos mitades deben mantenerse estrictamente separadas (ver sección 4.6 y sección 8).

1.3 Público objetivo

- Desarrolladores individuales que quieren una retrospectiva anual o periódica de su actividad en GitHub.
- Equipos o comunidades técnicas que quieren compartir su actividad de forma visual (por ejemplo, en redes sociales o portafolios).
- Usuarios con cuentas públicas de GitHub; en el MVP no se contempla analizar organizaciones ni repositorios privados.

1.4 Alcance del proyecto

Dentro del alcance (MVP)

- Autenticación vía GitHub OAuth y sincronización de datos públicos del usuario autenticado.
- Análisis de repositorios públicos del usuario: commits, lenguajes, calendario de contribuciones.
- Cálculo de métricas de comportamiento normalizadas a la zona horaria del usuario.
- Generación de insights narrativos a partir de esas métricas mediante IA.
- Experiencia “Wrapped” navegable y un dashboard de datos.
- Mecanismo de compartición del reporte generado.

Fuera del alcance:

- Análisis de repositorios privados (función futura, de tipo opt-in explícito).
- Gamificación, comparaciones entre usuarios y features de IA conversacional.
- Soporte para organizaciones o cuentas de equipo.

2. ARQUITECTURA DEL SISTEMA

El sistema se organiza en capas con responsabilidades estrictamente separadas. Ninguna capa debe asumir responsabilidades de otra: esto es lo que permite que el Analytics Engine y el Insights Engine se puedan testear de forma aislada, sin red ni base de datos real, y que la lógica de negocio sobreviva a cambios en la fuente de datos (por ejemplo, si en el futuro se añaden otras plataformas además de GitHub).

2.1 Diagrama de flujo de datos

```
GitHub (REST + GraphQL)
        |
Data Collector        <- habla con GitHub, maneja rate limits y retry/backoff
        |
Job Queue             <- procesamiento async, fuera del ciclo de vida de una request
        |
Database              <- PostgreSQL vía Prisma
        |
Analytics Engine      <- independiente de GitHub API, de la UI y de la DB
        |
Insights Engine       <- IA narra métricas ya calculadas, nunca las calcula
        |
Dashboard / Wrapped / Sharing
```

2.2 Responsabilidad de cada capa

Capa	Responsabilidad	No debe hacer
Data Collector	Autenticar llamadas a GitHub, paginar resultados, manejar rate limiting y aplicar retry con backoff exponencial.	Calcular métricas ni decidir qué es “actividad relevante”.
Job Queue	Ejecutar sincronizaciones pesadas de forma asíncrona, con reintentos e idempotencia.	Contener lógica de negocio compleja; solo orquesta.
Database	Persistir entidades normalizadas (usuarios, repos, commits, stats de lenguaje, reportes).	Servir como motor de cálculo analítico.
Analytics Engine	Transformar datos crudos en métricas estructuradas y verificables, normalizadas a la zona horaria del usuario.	Depender de la red, de Prisma directamente o de componentes de UI.
Insights Engine	Convertir métricas estructuradas en lenguaje natural mediante IA.	Calcular o inferir estadísticas; su entrada ya debe venir calculada.
Dashboard / Wrapped / Sharing	Presentar los datos y los insights; generar artefactos compartibles.	Contener lógica de cálculo analítico.

2.3 Principios arquitectónicos

- **Separación estricta de capas:** cada capa se comunica solo con la capa inmediatamente adyacente en el flujo de datos.
- **Testabilidad sin dependencias externas:** el Analytics Engine y el Insights Engine deben poder testearse con datos en memoria, sin red ni base de datos real. Esto implica que ambos reciben sus entradas como parámetros (inyección de dependencias), nunca consultando directamente a Prisma o a fetch.
- **Procesamiento asíncrono por defecto:** cualquier operación cuya duración dependa del volumen de datos del usuario (número de commits, de repos) se ejecuta en la Job Queue, nunca de forma síncrona dentro de una request HTTP.

- **La base de datos es el árbitro de consistencia:** la deduplicación y el control de concurrencia se resuelven contra el estado persistido, no con patrones de verificar-y-luego-escribir en memoria.

3. STACK TECNOLÓGICO

3.1 Frontend

Tecnología	Rol en el proyecto
Next.js (App Router)	Framework principal; enrutamiento, renderizado híbrido (server/client components) y Route Handlers / Server Actions para el backend ligero.
React	Librería de UI sobre la que se construye Next.js.
TypeScript	Tipado estático en todo el proyecto, incluyendo el esquema de Prisma y los contratos entre capas.
Tailwind CSS	Sistema de estilos utilitario para el dashboard y la experiencia Wrapped.

3.2 Visualización de datos

Se utiliza Recharts o Chart.js para renderizar las métricas del Analytics Engine (calendario de contribuciones, evolución de lenguajes, distribución horaria de actividad, rachas). La elección entre una u otra librería depende de la necesidad de composición declarativa con componentes React (a favor de Recharts) frente a mayor control imperativo sobre el canvas (a favor de Chart.js).

3.3 Backend

La lógica ligera —lectura de datos ya persistidos, validaciones, disparo de jobs— se implementa en Next.js mediante Route Handlers y Server Actions. Se considera “limpia” una operación cuya duración es independiente del volumen de datos del usuario; cualquier operación cuya duración escale con el número de commits o repositorios se delega a la Job Queue (ver sección 3.4 y sección 4.2).

3.4 Jobs en segundo plano

Candidatos evaluados: Inngest, Trigger.dev o QStash. Los tres cumplen el requisito central: ejecutar trabajo fuera del ciclo de vida de una función serverless de Vercel, con reintentos y control de concurrencia.

Opción	Fortalezas a considerar
Inngest	Modelo basado en eventos y funciones durables (steps), con reintentos automáticos y buen soporte para flujos con múltiples pasos (ej. sincronizar repos → sincronizar commits → generar reporte).
Trigger.dev	Orientado a tareas largas con observabilidad integrada; cómodo para trabajos con muchas llamadas externas secuenciales.
QStash	Modelo más simple de cola/mensaje con reintentos vía HTTP; menor superficie de API, útil si se prefiere minimizar dependencias.

Restricción no negociable: *independientemente de la opción elegida, nunca se debe usar un Route Handler síncrono para sincronización pesada (ver sección 6).*

3.5 Base de datos y ORM

- Base de datos: PostgreSQL (por ejemplo, en Neon u otro proveedor administrado compatible con Vercel).
- ORM: Prisma, que provee el cliente tipado, las migraciones y el adapter requerido por Auth.js.

3.6 Autenticación

Se utiliza Auth.js con proveedor de GitHub OAuth y el adapter de Prisma, de modo que las sesiones, cuentas y tokens de verificación se persisten en la misma base de datos del proyecto (ver flujo completo en la sección 5).

3.7 Deployment

El despliegue se realiza en Vercel. Esta decisión es la que origina directamente la restricción de no ejecutar sincronización pesada en un Route Handler: las funciones serverless de Vercel tienen un límite de tiempo de ejecución, y un usuario con decenas de miles de commits no cabe en una sola solicitud HTTP.

4. INTEGRACIÓN CON LA API DE GITHUB

4.1 GitHub REST API

Se utiliza para los siguientes recursos:

Recurso	Endpoint de referencia	Uso en el proyecto
Usuario autenticado	GET /user	Obtener perfil básico tras el login OAuth.
Repositorios del usuario	GET /user/repos	Listar repositorios públicos a sincronizar (paginado).
Commits por repositorio	GET /repos/{owner}/{repo}/commits	Obtener el historial de commits de cada repositorio (paginado, filtrable por autor y por fecha).
Lenguajes por repositorio	GET /repos/{owner}/{repo}/languages	Obtener el desglose de bytes de código por lenguaje en cada repositorio.

4.2 GitHub GraphQL API

Se utiliza para el contribution calendar y para cualquier consulta que combine varios recursos en una sola llamada. El contribution calendar no existe como endpoint en la API REST, por lo que su uso mediante GraphQL es obligatorio. Consulta de referencia:

```
query ContributionCalendar($login: String!, $from: DateTime!, $to: DateTime!) {
  user(login: $login) {
    contributionsCollection(from: $from, to: $to) {
      contributionCalendar {
        totalContributions
        weeks {
          contributionDays {
            date
            contributionCount
          }
        }
      }
    }
  }
}
```

4.3 Manejo de rate limits

Los límites de REST (5.000 solicitudes por hora) y de GraphQL (sistema de puntos, informado en el campo cost de cada respuesta) son independientes entre sí y deben gestionarse con contadores y lógica propios para cada cliente, nunca con un único presupuesto compartido.

API	Mecanismo de límite	Señal a monitorear
REST	5.000 solicitudes por hora por token.	Cabeceras X-RateLimit-Remaining y X-RateLimit-Reset.
GraphQL	Sistema de puntos por consulta, con presupuesto propio por hora.	Campo rateLimit.cost y rateLimit.remaining en la respuesta.

4.4 Retry con backoff exponencial

Toda llamada a la API de GitHub que pueda devolver 403 (rate limit) o 429 (too many requests) debe reintentarse con backoff exponencial. Esquema de referencia:

```
async function fetchWithBackoff(request, maxRetries = 5) {
  for (let attempt = 0; attempt <= maxRetries; attempt++) {
    const response = await request();
    if (response.status !== 403 && response.status !== 429) {
      return response;
    }
    const resetHeader = response.headers.get('x-ratelimit-reset');
    const waitMs = resetHeader
      ? Math.max(0, Number(resetHeader) * 1000 - Date.now())
      : 2 ** attempt * 1000; // backoff exponencial
    await sleep(waitMs);
  }
  throw new Error('GitHub rate limit excedido tras reintentos');
}
```

5. AUTENTICACIÓN Y AUTORIZACIÓN

5.1 Flujo OAuth con GitHub

1. El usuario inicia sesión desde la aplicación y es redirigido al flujo de autorización de GitHub OAuth.
2. GitHub solicita al usuario que autorice los scopes requeridos por la aplicación.
3. GitHub redirige de vuelta a la aplicación con un código de autorización.
4. Auth.js intercambia ese código por un access token (y, si aplica, un refresh token) mediante el proveedor de GitHub.
5. El adapter de Prisma persiste la cuenta, la sesión y los tokens asociados al usuario.
6. El accessToken se cifra antes de guardarse en el modelo GitHubAccount (ver sección 10).

5.2 Scopes recomendados

- read:user — para obtener el perfil del usuario autenticado.
- user:email — para resolver los correos verificados de la cuenta y así identificar correctamente la autoría de los commits (ver sección 7.3).
- public_repo (o el acceso de solo lectura equivalente a repositorios públicos) — para listar y leer repositorios públicos del usuario.

Principio de mínimo privilegio: *no deben solicitarse scopes de escritura ni acceso a repositorios privados en el MVP, consistente con la restricción de alcance de la sección 1.4.*

6. MODELO DE DATOS (PRISMA)

El esquema se organiza en torno a seis grupos de entidades: identidad del usuario, autenticación (Auth.js), conexión con GitHub, datos sincronizados y el reporte final.

User

Campo	Tipo	Notas
id	String (cuid/uuid)	Identificador primario.
name / email / image	String?	Perfil estándar, provisto por Auth.js.
timezone	String	Zona horaria IANA del usuario (ej. America/Bogota). Obligatoria para toda la analítica temporal (ver sección 8.2).
createdAt	DateTime	Fecha de creación del registro.

Account, Session, VerificationToken

Modelos estándar requeridos por el adapter de Prisma de Auth.js para la gestión de cuentas, sesiones y tokens de verificación. No se modifican respecto al esquema estándar del adapter.

GitHubAccount

Campo	Tipo	Notas
userId	String	Relación con User.
accessToken	String (cifrado)	Nunca se persiste en texto plano (ver sección 10.1).
refreshToken	String? (cifrado)	Si el flujo OAuth lo provee.
scope	String	Scopes efectivamente otorgados por el usuario.
tokenType	String	Tipo de token (normalmente bearer).
expiresAt	DateTime?	Fecha de expiración, cuando aplica.

Repository

- Identificador de GitHub (githubId), nombre completo (owner/repo), visibilidad, lenguaje principal y relación con el User propietario.

Commit

Campo	Tipo	Notas
sha	String	Identificador del commit en GitHub.
repositoryId	String	Relación con Repository.
authorEmail	String	Usado para la resolución de identidad del autor contra los correos verificados del usuario (ver sección 7.3).
committedAtUtc	DateTime	Timestamp del commit en UTC, tal como lo entrega GitHub.

Nota: *el timestamp se almacena siempre en UTC; la conversión a la hora local del usuario ocurre exclusivamente en el Analytics Engine, nunca en la capa de persistencia (ver sección 8.2).*

LanguageStat

Es una serie de tiempo, no un snapshot único: cada fila incluye un campo capturedAt además del repositorio y el desglose de lenguajes en ese momento. Este diseño es el que permite construir la funcionalidad de “Language Evolution” (evolución del uso de lenguajes a lo largo del tiempo); un modelo de snapshot único haría imposible esa métrica.

WrappedReport

Representa el reporte final generado para el usuario: el conjunto de métricas calculadas por el Analytics Engine junto con los insights narrativos producidos por el Insights Engine, listos para la Wrapped Experience y para compartir.

7. DATA COLLECTOR

El Data Collector es la única capa que habla directamente con la API de GitHub. Encapsula tres responsabilidades: autenticación de las llamadas con el accessToken del usuario, paginación de resultados (repositorios y commits pueden abarcar cientos de páginas) y el manejo de rate limiting y retry/backoff descrito en la sección 4.3 y 4.4.

7.1 Paginación

Tanto la lista de repositorios como la de commits por repositorio están paginadas en la API REST. El Data Collector debe iterar todas las páginas disponibles antes de entregar el resultado a la Job Queue, respetando en cada iteración el presupuesto de rate limit restante.

7.2 Separación de responsabilidades

El Data Collector no decide qué hacer con los datos obtenidos ni calcula métricas: solo los normaliza al formato esperado por los modelos de Prisma y los entrega a la capa siguiente.

7.3 Resolución de identidad del autor

La identidad del autor de un commit se resuelve contra los correos electrónicos verificados de la cuenta del usuario (obtenidos vía el scope user:email), no únicamente contra el campo author.login del commit. Esto es necesario porque un mismo usuario puede haber hecho commits con distintas configuraciones locales de Git (por ejemplo, un correo personal en un repo y un correo corporativo en otro) que no siempre coinciden con su username de GitHub. Resolver solo por login subestimaría la actividad real del usuario.

8. JOB QUEUE

Toda sincronización pesada se ejecuta a través de una cola de trabajos, nunca dentro de un Route Handler síncrono. Esta es una restricción no negociable del proyecto (ver sección 11), motivada directamente por el límite de tiempo de ejecución de las funciones serverless de Vercel.

8.1 Tipos de jobs

Job	Disparador	Qué hace
syncUserRepositories	Login inicial o sincronización manual.	Obtiene la lista de repositorios públicos del usuario vía el Data Collector y los persiste.
syncRepositoryCommits	Por cada repositorio sincronizado.	Obtiene los commits del repositorio (con resolución de identidad) y los persiste.
syncLanguageStats	Por cada repositorio sincronizado.	Obtiene el desglose de lenguajes y agrega una nueva fila a la serie de tiempo LanguageStat.
syncContributionCalendar	Al finalizar la sincronización de repos.	Consulta el contribution calendar vía GraphQL.
generateWrappedReport	Al completarse la sincronización.	Invoca al Analytics Engine y al Insights Engine para producir el WrappedReport.

8.2 Idempotencia y deduplicación

Cada job debe poder re-ejecutarse sin duplicar datos: la deduplicación se resuelve siempre contra el estado persistido en la base de datos (por ejemplo, un upsert por sha de commit), nunca con comprobaciones en memoria del tipo “verificar y luego escribir”, que son vulnerables a condiciones de carrera entre ejecuciones concurrentes del mismo job.

9. ANALYTICS ENGINE

El Analytics Engine transforma los datos persistidos en métricas estructuradas. Debe ser independiente de la API de GitHub, de la interfaz de usuario y, en la medida de lo posible, de la base de datos: recibe colecciones de datos ya normalizados (por ejemplo, un arreglo de commits con su timestamp en UTC) y devuelve métricas, sin depender de Prisma ni de fetch directamente. Esto es lo que permite testearlo con fixtures en memoria.

9.1 Normalización de zona horaria

Todas las métricas temporales (día más activo, hora más activa, rachas) deben calcularse sobre la hora local del usuario, nunca en UTC. Los timestamps se almacenan en UTC (ver sección 6, modelo Commit) y se convierten a la zona horaria del usuario (campo timezone del modelo User) en el momento de la agregación, usando una librería con soporte de IANA timezones y horario de verano (por ejemplo, date-fns-tz o Luxon).

```
function findMostActiveHour(commits, timezone) {

  const counts = new Array(24).fill(0);

  for (const commit of commits) {

    const localHour = getHourInTimeZone(commit.committedAtUtc, timezone);

    counts[localHour]++;

  }

  return counts.indexOf(Math.max(...counts));

}
```

9.2 Métricas calculadas

- Día de la semana más activo y hora del día más activa, normalizados a la zona horaria local.
- Rachas de actividad (streaks) consecutivas, en días locales del usuario.
- Evolución del uso de lenguajes a lo largo del tiempo, a partir de la serie LanguageStat.
- Calendario de contribuciones, obtenido vía GraphQL (sección 4.2).
- **Developer Activity Score:** métrica compuesta que resume el nivel de actividad del usuario. Se nombra explícitamente “Developer Activity Score” y nunca “Productivity Score”, porque un mayor número de commits no implica mayor productividad ni mayor calidad de código; el nombre debe dejar claro que mide actividad, no desempeño.

9.3 Testabilidad

Las funciones del Analytics Engine, incluyendo findMostActiveDay() y findMostActiveHour(), deben tener tests unitarios que cubran explícitamente casos con usuarios en distintas zonas horarias, incluyendo casos límite alrededor de cambios de horario de verano (DST) y de medianoche local.

10. INSIGHTS ENGINE

El Insights Engine es la capa que convierte las métricas estructuradas del Analytics Engine en lenguaje natural. El pipeline correcto es estrictamente unidireccional:

```
datos -> Analytics Engine -> metricas estructuradas -> IA -> lenguaje natural
```

Nunca:

```
datos -> IA -> estadisticas
```

La inteligencia artificial se usa exclusivamente para interpretar y redactar; nunca para calcular estadísticas. Esto significa que el prompt enviado al modelo de IA debe contener únicamente números y hechos ya calculados y verificados (por ejemplo, “el usuario hizo 42 commits los martes a las 9pm hora local, su racha más larga fue de 14 días”), nunca datos crudos sin procesar de los cuales se le pida inferir cifras.

Esta separación es la que garantiza que las cifras mostradas al usuario sean siempre exactas y reproducibles, independientemente de la calidad o las alucinaciones del modelo de lenguaje: en el peor caso, una narración deficiente afecta el tono del reporte, nunca la veracidad de un número.

11. DASHBOARD, WRAPPED EXPERIENCE Y SHARING

11.1 Dashboard

Vista de consulta de los datos ya sincronizados y de las métricas del Analytics Engine, pensada para exploración libre por parte del usuario (fuera del formato narrativo del Wrapped).

11.2 Wrapped Experience

Experiencia narrativa, tipo presentación por pasos (“slides”), que combina las métricas calculadas con los insights redactados por el Insights Engine, usando las visualizaciones descritas en la sección 3.2.

11.3 Sharing

Mecanismo para compartir el reporte generado (por ejemplo, como imagen exportable o enlace público de solo lectura), de modo que el usuario pueda difundir su Wrapped fuera de la aplicación.

12. RESTRICCIONES NO NEGOCIABLES

Estas reglas aplican a todo el proyecto y no admiten excepciones dentro del alcance de la aplicación:

1. **Alcance — solo repositorios públicos.** El análisis de repositorios privados es una función futura, de tipo opt-in, y no forma parte del alcance inicial.
2. **Sincronización pesada siempre por Job Queue.** Nunca mediante un Route Handler síncrono. Las funciones serverless de Vercel tienen un límite de tiempo de ejecución, y un usuario con decenas de miles de commits no cabe en una sola solicitud.
3. **Zona horaria local del usuario.** Toda métrica temporal se calcula sobre la hora local, nunca en UTC. Requiere el campo timezone en User y normalización de cada timestamp antes de agregarlo.

4. **Resolución de identidad por correos verificados.** La identidad del autor de un commit se resuelve contra los correos verificados de la cuenta, no solo contra author.login, para no subestimar la actividad real.
5. **La IA nunca calcula estadísticas.** Solo interpreta y redacta a partir de métricas ya estructuradas por el Analytics Engine.
6. **Tokens de GitHub siempre cifrados en reposo.** Nunca en texto plano ni expuestos en logs.

13. SEGURIDAD

- **Cifrado de tokens en reposo:** los campos accessToken y refreshToken del modelo GitHubAccount se cifran antes de persistirse (por ejemplo, con AES-256-GCM y una clave gestionada fuera del repositorio) y se descifran únicamente en el momento de uso por el Data Collector.
- **Nunca en logs:** ningún token, header de autorización ni variable de entorno sensible debe aparecer en logs de aplicación, de build o de CI/CD.
- **Mínimo privilegio en scopes OAuth:** solo se solicitan los scopes estrictamente necesarios para leer datos públicos (ver sección 5.2).
- **Datos únicamente públicos en el MVP:** al no procesarse repositorios privados, la superficie de exposición de datos sensibles del usuario se mantiene deliberadamente acotada.

14. CONVENCIONES DE DESARROLLO Y TESTING

- Retry con backoff exponencial en toda llamada a la API de GitHub que pueda devolver 403 o 429 (ver sección 4.4).
- Tests de integración con mocking de HTTP (por ejemplo, msw); nunca se ejecutan contra la API real de GitHub en CI.
- Los tests unitarios de findMostActiveDay() y findMostActiveHour() deben incluir explícitamente casos con usuarios en distintas zonas horarias.
- La métrica compuesta del proyecto se nombra siempre “Developer Activity Score”, nunca “Productivity Score”.
- El Analytics Engine y el Insights Engine se diseñan con inyección de dependencias para poder testearse sin red ni base de datos real (ver sección 2.3).

15. ORDEN DE PRIORIDAD DE CONSTRUCCIÓN

El proyecto se construye siguiendo el siguiente orden de prioridad, de mayor a menor, cada uno con un criterio de “listo” (DoD) orientativo:

#	Entregable	Criterio de “listo” orientativo
1	Cliente de la API de GitHub (REST + GraphQL) y rate limiting	Cliente tipado con manejo independiente de límites REST/GraphQL y retry con backoff (sección 4).
2	OAuth y modelos de Auth.js	Login funcional con GitHub, sesión persistida, scopes mínimos otorgados (sección 5).
3	Data Collector	Sincroniza repos, commits y lenguajes con paginación y resolución de identidad (sección 7).
4	Job Queue	Jobs idempotentes para cada tipo de sincronización (sección 8).
5	Base de datos	Esquema Prisma completo y migraciones aplicadas (sección 6).

6	Analytics Engine (con normalización de timezone)	Métricas correctas y testeadas en múltiples zonas horarias (sección 9).
7	Dashboard	Visualización de métricas ya calculadas (sección 11.1).
8	Wrapped Experience	Experiencia narrativa navegable (sección 11.2).
9	Insights Engine	Narración generada solo a partir de métricas estructuradas (sección 10).
10	Sharing	Mecanismo de compartición funcional (sección 11.3).
11	Gamificación / Comparaciones / IA conversacional	Etapa posterior, fuera del MVP.
12	Repositorios privados como opt-in	Etapa posterior, fuera del MVP.

16. RIESGOS Y CONSIDERACIONES ABIERTAS

- **Usuarios con historiales muy grandes:** decenas de miles de commits pueden tensionar tanto el rate limit de GitHub como el tiempo de los jobs; la estrategia de paginación y backoff debe probarse con cuentas de alto volumen.
- **Cambios de horario de verano (DST):** los cálculos de día/hora más activa y de rachas deben validarse específicamente alrededor de transiciones de DST en las zonas horarias soportadas.
- **Cambios en la API de GitHub:** tanto el esquema de GraphQL como los campos de REST pueden cambiar; el Data Collector debe aislar estos detalles para minimizar el impacto en capas superiores.
- **Costo y latencia de la IA narrativa:** la generación de insights depende de un proveedor externo de IA; conviene cachear o persistir el resultado en WrappedReport para no regenerar narrativa en cada visualización.
- **Cuentas con poca o nula actividad:** el Analytics Engine y el Insights Engine deben manejar explícitamente el caso de un usuario con muy pocos commits, sin producir métricas engañosas ni narrativas vacías.